

Algoritmos e Estruturas de Dados II

Corretude de algoritmos iterativos e invariantes de laço — Parte 2

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe
andreyoshiaki@dcomp.ufs.br

Relembrar o método: invariante + inicialização, manutenção e término



Prova de corretude da **busca binária** — e por que tantas implementações erram



Prova de corretude do **particionamento de Lomuto** (coração do quicksort)



Usar o invariante como **ferramenta de projeto**, não só de prova

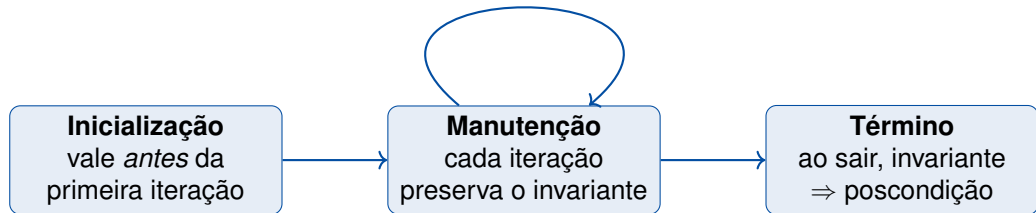
Relembrando o método

Terceiro algoritmo: busca binária

Quarto algoritmo: particionamento de Lomuto

Invariante como ferramenta de projeto

Relembrando o método



- Na Parte 1: busca do máximo (invariante sobre um valor) e ordenação por inserção (invariante estrutural, laços aninhados).
- Hoje: busca binária e particionamento — e, para corretude total, uma medida inteira não negativa que decresce a cada iteração (garantia de término).

Terceiro algoritmo: busca binária

Problema:

Dado $A[0..n-1]$ **ordenado** e um valor x , devolver um índice i com $A[i] = x$, ou -1 se x não está no vetor.

- Publicada em 1946; a primeira versão correta para todo n demorou mais de uma década a aparecer.
- Bentley: a maioria dos programadores profissionais não a implementa corretamente de primeira.

Solução:

Manter uma *janela* $[lo..hi]$ onde x *ainda pode* estar e cortá-la pela metade a cada comparação.

- O invariante diz exatamente o que a janela significa — e dita como atualizar lo e hi sem erro.

```
1  int busca_binaria(int A[], int n, int x) { /* pre: A ordenado */
2      int lo = 0, hi = n - 1;
3      while (lo <= hi) {
4          /* invariante: se x esta em A, esta em A[lo..hi] */
5          int meio = lo + (hi - lo) / 2;
6          if (A[meio] == x)      return meio;
7          else if (A[meio] < x)  lo = meio + 1;
8          else                  hi = meio - 1;
9      }
10     return -1; /* janela vazia: x nao esta no vetor */
11 }
```

Invariante

No início de cada iteração: **se** x ocorre em $A[0..n - 1]$, **então** x ocorre em $A[lo..hi]$.



- Cinza: regiões já **descartadas** — a ordenação garante que x não está nelas.
- O invariante é *condicional*: não promete que x está na janela; promete que **fora dela é impossível**.
- Janela vazia ($lo > hi$) + invariante $\Rightarrow x$ não está no vetor: o return -1 está *provado*, não “chutado”.

- **Inicialização:** $[lo..hi] = [0..n - 1]$ é o vetor inteiro — a implicação vale trivialmente. ✓
- **Manutenção:** seja m o meio. Se $A[m] < x$, a ordenação dá $A[k] \leq A[m] < x$ para todo $k \leq m$: descartar $[lo..m]$ é seguro, e $lo = m + 1$ preserva o invariante. Caso $A[m] > x$ é simétrico. ✓
- **Término com retorno no laço:** só retornamos $meio$ quando $A[meio] = x$ — correto por inspeção. ✓
- **Término com janela vazia:** $lo > hi$ e o invariante (contrapositiva): x não está em $A[lo..hi]$ vazio, logo não está no vetor — devolver -1 é correto. ✓
- **O laço termina?** A medida $hi - lo + 1$ (tamanho da janela) é ≥ 0 e **decresce estritamente**: $meio$ está em $[lo..hi]$, e os dois ramos excluem $meio$ da nova janela. ✓

Erro 1: $lo = meio$ em vez de $lo = meio + 1$

Com janela $[3..4]$, $meio = 3$ e $A[3] < x$: a “nova” janela continua $[3..4]$. A medida $hi - lo + 1$ **não decresce** — laço infinito. A prova de término exige excluir $meio$.

Erro 2: $meio = (lo + hi) / 2$

Matematicamente idêntico — mas em C, com n grande, $lo + hi$ pode estourar o `int` (overflow) e tornar $meio$ negativo. $lo + (hi - lo) / 2$ calcula o mesmo valor sem estourar.

- O primeiro erro é *lógico*: a prova o captura no passo de término.
- O segundo é da *máquina*: a prova vale sobre inteiros ideais; a implementação precisa respeitar os limites do tipo.

O que aprendemos

- Invariante condicional: “se x está no vetor, está na janela $[lo..hi]$ ”.
- A manutenção usa a **precondição** (vetor ordenado) para justificar o descarte.
- Término via medida decrescente: o tamanho da janela cai a cada iteração — e é isso que proíbe `lo = meio`.
- O caso “não encontrado” sai *provado* do invariante, de graça.

Quarto algoritmo: particionamento de Lomuto

Problema:

Reorganizar $A[p..r]$ em torno do pivô $A[r]$: menores ou iguais à esquerda, maiores à direita, pivô no meio — devolvendo a posição final do pivô.

- É o coração do **quicksort**: se o particionamento está certo, a recursão faz o resto.

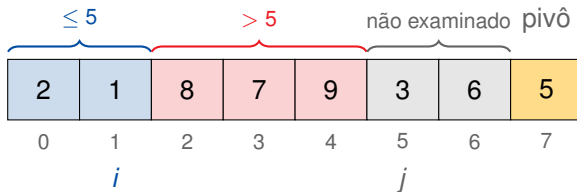
Solução:

Varrer o trecho mantendo **quatro regiões** bem delimitadas por dois índices (i e j).

- O invariante aqui é uma *fotografia* das quatro regiões — o exemplo mais visual dos quatro algoritmos que estudamos.

```
1  int particiona(int A[], int p, int r) {
2      int pivo = A[r];
3      int i = p - 1;
4      for (int j = p; j < r; j++) {
5          /* invariante: A[p..i] <= pivo, */
6          /* A[i+1..j-1] > pivo, A[j..r-1] nao examinado */
7          if (A[j] <= pivo) {
8              i++;
9              troca(&A[i], &A[j]);
10         }
11     }
12     troca(&A[i + 1], &A[r]);
13     return i + 1;
14 }
```

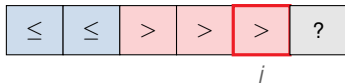
O invariante das quatro regiões



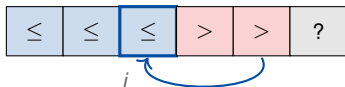
Invariante (início da iteração j)

$A[p..i] \leq \text{pivô}$ e $A[i + 1..j - 1] > \text{pivô}$ e $A[j..r - 1]$ não examinado e $A[r] = \text{pivô}$.

Caso $A[j] > \text{pivô}$: só avançar j — a região vermelha cresce.



Caso $A[j] \leq \text{pivô}$: $i++$ e trocar $A[i] \leftrightarrow A[j]$ — a azul cresce.



- A troca move o *primeiro* elemento da região vermelha para o fim dela — as duas regiões continuam homogêneas; se a vermelha está vazia ($i + 1 = j$), a “troca” é de $A[j]$ com ele mesmo e o invariante segue valendo.

- **Inicialização** ($j = p$, $i = p - 1$): as regiões $A[p..p - 1]$ e $A[p..p - 1]$ são vazias — as duas cláusulas valem por vacuidade. ✓
- **Manutenção**: os dois casos do slide anterior preservam as quatro regiões. ✓
- **Término**: com $j = r$, todo $A[p..r - 1]$ está classificado: $A[p..i] \leq \text{pivô}$ e $A[i + 1..r - 1] > \text{pivô}$. A troca final põe o pivô em $A[i + 1]$, entre as duas regiões — exatamente a poscondição, com retorno $i + 1$. ✓
- **O laço termina?** É um `for` com j de p a $r - 1$: a medida $r - j$ decresce a cada iteração. ✓
- **Permutação**: só fazemos trocas — nenhum elemento é criado ou perdido (mesma cláusula que na ordenação por inserção).

Exemplo: rastreando $A = [3, 8, 1, 5]$ com pivô 5

Início de j	i	Vetor	Regiões $[\leq > ?]$
$j = 0$	-1	$[3, 8, 1 \mid 5]$	$[\mid \mid 3, 8, 1]$
$j = 1$	0	$[3, 8, 1 \mid 5]$	$[3 \mid \mid 8, 1]$
$j = 2$	0	$[3, 8, 1 \mid 5]$	$[3 \mid 8 \mid 1]$
saída ($j = 3$)	1	$[3, 1, 8 \mid 5]$	$[3, 1 \mid 8 \mid]$
troca final	—	$[3, 1, 5, 8]$	pivô na posição 2 ✓

- Em $j = 2$: $A[2] = 1 \leq 5$, então i vira 1 e trocamos $A[1] \leftrightarrow A[2]$.
- A troca final coloca o pivô entre as regiões: tudo à esquerda ≤ 5 , tudo à direita > 5 .

O que aprendemos

- Invariante “geográfico”: o vetor dividido em quatro regiões com significados distintos.
- Inicialização por **vacuidade**: regiões vazias satisfazem qualquer propriedade.
- A manutenção é uma análise de dois casos, cada um fazendo uma região crescer.
- Com o particionamento provado, a corretude do quicksort segue por indução no tamanho do trecho.

Invariante como ferramenta de projeto

A inversão de perspectiva

Em vez de escrever o laço e *depois* procurar o invariante, escolha o invariante e deixe-o **ditar** o código.

- **Inicialização** → diz como declarar e inicializar as variáveis.
- **Manutenção** → diz o que o corpo do laço *precisa* fazer.
- **Término + poscondição** → diz qual é a condição de parada.

Nos quatro algoritmos que estudamos

Máximo: “maior já visto”. Inserção: “prefixo ordenado”.

Busca binária: “x só pode estar em $[lo..hi]$ ”. Lomuto: “quatro regiões”.

Algoritmo	Tipo de invariante	O que a prova exigiu
Busca do máximo	valor acumulado	caso base não trivial ($\text{max} = A[0]$)
Ordenação por inserção	prefixo estruturado	ordem + permutação; laços aninhados
Busca binária	janela condicional	precondição (ordenado) + medida decrescente
Particionamento	regiões geográficas	vacuidade + análise de casos

- O método é sempre o mesmo: **inicialização, manutenção, término** — o que muda é a criatividade para *enunciar* o invariante.
- Enunciar o invariante certo é a metade difícil — e é uma habilidade que se treina: o tutorial desta aula é esse treino.

-  **Jon Bentley.**
Programming Pearls.
Addison-Wesley, 2 edition, 2000.
-  **Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.**
Introduction to Algorithms.
MIT Press, 3 edition, 2009.
-  **David Gries.**
The Science of Programming.
Springer-Verlag, 1981.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.